

Enterprise Architecture Modernization Progress Log

Master Specification: ARCHITECTURE_REDESIGN_PLAN.md
Task Checklist: CHECKLIST.md
Historical Archive (Phases 0–9 TSPL Engine): PROGRESS_LEGACY.md

Log Format Standard

Each completed sub-task is logged with the following structure:

- **Sub-Task ID & Title:** Exact reference to the Master Plan.
- **Date Completed:** Timestamp.
- **What Was Done:** Files created or modified, symbols exported, and UI components updated.
- **How It Was Done:** Architectural design, mathematical models, algorithms, and Reanimated worklet / Skia bindings.
- **Verification & Quality Gate Results:** Output of tests, type checking (`tsc --noEmit`), and performance benchmarks.
- **Notes & Next Step:** Handoff context for the next sequential task.

Standalone Bugfixes (Legacy Konva Canvas — Non-Phase 1)

Bugfix: Vertical Resize Height Jitter on Barcode/Shape/Image Elements (Legacy Konva Canvas) — *Retroactive Entry*

- **Scope:** `src/lib/editor/label-bounds.ts` (`clampToLabelBounds`, anchor `'s'`), `src/components/editor/element-renderer.tsx`.
- **Date Completed:** 2026-09-18 (commit `0675688`). **Entry written:** 2026-09-23 during the documentation sync. The checklist item existed with no progress entry.
- **Problem & Root Cause (reconstructed from the commit diff):** In the `'s'` (bottom-handle) branch, `clampToLabelBounds` computed `targetH` and, when `top + targetH` exceeded the canvas height, moved `top` upward (`top = canvasH - targetH`). Dragging the south handle near the bottom edge therefore shifted the element's top edge on every frame, which showed as vertical jitter.
- **What Was Done:** `top` is now fixed during an `'s'` resize (`top = clamp(top, 0, canvasH - minMm)`), and height is clamped to the space below it (`maxAllowedHeight = canvasH - top`), with `overflowed = targetH > maxAllowedHeight`. The same commit also stopped the `'e'` branch adjusting vertical headroom unless `naturalHeight` is provided; that part is logged under "Barcode Width Scaling & HRI Text Distortion" below.
- **Verification & Quality Gate Results: None recorded.** No test output or on-device result was logged for this fix at the time. The later anchor audit in the next entry states that anchor `'s'` keeps `top / left / width` immutable, which covers the behaviour, but it was not recorded as verification of this item.

Bugfix: Barcode Width Scaling & HRI Text Distortion (Legacy Konva Canvas)

- **Scope:** Legacy Konva bounds clamping & element renderer (`src/lib/editor/label-bounds.ts`, `src/components/editor/element-renderer.tsx`).
- **Date Completed:** 2026-09-18
- **Problem & Root Cause:**
 - When resizing a Barcode element horizontally using the East ('e') handle, `clampToLabelBounds` unconditionally ran vertical headroom adjustment code (`targetH = Math.max(minMm, naturalH ?? height)`), which nudged `top` upward when elements were placed near the lower canvas edge even when `naturalHeight` was `undefined`.
 - In `BarcodeContent` within `src/components/editor/element-renderer.tsx`, the HRI text was not centered inside a full-width container and lacked clearance from the South resize handle, causing horizontal stretching distortion and text truncation.
- **Anchor Audit in `clampToLabelBounds`:**
 - Audited all anchor branches ('e', 's', 'body'):
 - Anchor 'e': Guarded vertical headroom reflow behind `if (naturalH !== undefined)`. Non-text elements now retain strictly immutable `top` and `height` during width drags.
 - Anchor 's': Confirmed `top`, `left`, and `width` remain strictly immutable during vertical height drags.
 - Anchor 'body': Confirmed translation preserves element dimensions while respecting canvas bleed limits.
- **What Was Done:**
 1. Updated `clampToLabelBounds` to enforce perpendicular axis immutability for 'e' anchor when `naturalH === undefined`.
 2. Updated `BarcodeContent` in `element-renderer.tsx` to render HRI text with full width (100%), center alignment, clean flex layout, and handle clearance padding.
- **Verification & Quality Gate Results:**
 - `npx tsc --noEmit`: 0 errors.
 - `npm run test:labelx`: 100% PASS.
 - In-App Verification: Barcode element width stretches smoothly without vertical jumping, and HRI label text stays centered and legible without South handle overlap.

Bugfix & Real-Time UX: Real-Time Ruler Scale Highlighting During Move & Resize Gestures

- **Scope:** Editor ruler chrome & transform gesture engine (`src/components/canvas-rulers.tsx`, `src/app/edit.tsx`, `src/components/editor/konva-transformer.tsx`, `src/components/editor/konva-canvas.tsx`, `src/components/editor/skia-canvas.tsx`).
- **Date Completed:** 2026-09-18
- **Problem & Root Cause:**
 - Ruler scale highlighting (the projection band on horizontal and vertical rulers indicating the selected element's millimeter position and dimensions) previously only updated when the gesture was committed (`onTransformEnd` / finger drop).
 - This occurred because `HorizontalRuler` and `VerticalRuler` were reading solely from static React state (`selectedElement.left, top, width, height`), which is deferred during 1:1 drag gestures to prevent expensive React re-renders of the component tree.
- **What Was Done & Architectural Fix:**
 1. Reanimated `LiveRulerBounds` Shared Values:

- Defined `LiveRulerBounds` containing shared values for `leftMm`, `topMm`, `widthMm`, `heightMm`, and `visible`.
- In `src/app/edit.tsx`, initialized `liveRulerBounds` and synced them with React selection state when idle.

2. UI-Thread Animated Ruler Projection Overlay:

- Updated `HorizontalRuler` and `VerticalRuler` in `src/components/canvas-rulers.tsx` to render an `Animated.View` selection overlay driven by `useAnimatedStyle`.
- The overlay computes continuous millimeter-to-pixel projection on the GPU compositor at 120 fps without causing SVG tick or label re-renders.

3. Direct Worklet Streaming in Transformers:

- In `src/components/editor/konva-transformer.tsx` and `src/components/editor/skia-canvas.tsx`, gesture worklets directly write clamped coordinates to `liveBounds` shared values on every frame during both translation drag (`bodyDragGesture`) and resize (`createHandleGesture`, `widthResizeGesture`, `heightResizeGesture`).

• Verification & Quality Gate Results:

- `npx tsc --noEmit`: 0 errors.
- `npm run test:labelx`: 100% PASS.

Bugfix & Performance: QR Code Single-Path Vector Rendering & Fluid 120fps Gesture Scaling (Legacy Konva Canvas)

- **Scope:** Legacy Konva element renderer & transformer (`src/components/editor/element-renderer.tsx`, `src/components/editor/konva-transformer.tsx`, `src/lib/editor/resize-policy.ts`).
- **Checklist mapping:** This single entry is the record for the checklist item previously titled "QR Code Vector Rendering & Resize Bleed". The two were merged on 2026-09-23 (see the Documentation Sync entry at the end of this log): one change set, one date, one verification run. "Resize bleed" is item 2 below (headroom clamping).
- **Date Completed:** 2026-09-18
- **Problem & Root Cause:**
 - **Lag & Viewport Jank:** When resizing a QR element, `QrcodeContent` was rendering 400–600 individual `<Rect>` React Native SVG nodes. Continuously resizing a container with hundreds of native SVG subviews caused heavy native layout recalculations on every frame on Android/iOS.
 - **Bridge Messaging Congestion:** `createHandleGesture` was invoking `runOnJS(updateTooltipJS)` across the React Native bridge on every single touch event (120 fps).
 - **Origin Jitter / Drift:** `createHandleGesture` for `behavior === 'square'` previously computed dynamic vertical/horizontal midpoint offsets $(\text{originH} - \text{targetH}) / 2$, causing the top position to jump and fight touch pointer movements.
- **What Was Done & Architectural Fix:**
 1. **Single-Path High-Performance SVG Rendering (`element-renderer.tsx`):**
 - Replaced hundreds of individual `<Rect>` JSX elements with a single memoized SVG `<Path d={qrPath} fill={color} />` for QR codes, barcodes, and 2D matrices.

- Reduced native SVG shadow tree complexity by 99.8% (from 400+ nodes to 1 node), achieving instant GPU rasterization with zero layout cost during live animations.
- 2. **Pinned-Origin Square & Aspect Worklet Math** (**`konva-transformer.tsx`**, **`resize-policy.ts`**):
 - Pinned **`top`** and **`left`** strictly to **`(start.left, start.top)`** in both gesture worklets and **`boundingBoxMm`**.
 - Removed legacy midpoint relocation formulas **`(start.top + (start.height - height) / 2)`** and **`start.left + (start.width - width) / 2`** that previously shifted elements' positions upon resize.
 - Enforced available headroom clamping: **`maxAvailable = Math.min(canvasWPx - originLeft, canvasHPx - originTop)`**.
- 3. **Worklet Bridge Throttling** (**`konva-transformer.tsx`**):
 - Added **`lastTooltipTimeSv`** to throttle JS-bridge tooltip dispatches to once every 80ms, eliminating bridge message queuing while keeping UI-thread gesture animations running at full 120 fps.
- **Verification & Quality Gate Results:**
 - **`npx tsc --noEmit`**: 0 errors.
 - **`npm run test:labelx`**: 100% PASS.
 - **`src/lib/editor/__tests__/skia-scaling-engine.test.ts`**: QR 25×25 matrix parity, square-lock, and **`boundingBoxMm`** origin pinning all 100% PASS.
 - In-App Verification: Elements retain their exact origin coordinates when resizing via East and South handles; zero jumping, drift, or relocation upon release.

Phase 1: Interactive Editor Canvas & Resizing Engine (Editor UI & Geometry)

*Status: Complete, with scope reversal. Task 1.4 (2026-09-18) kept the live editor on Konva and made Skia print-only. Tasks 1.1–1.2 are **superseded**: the entry below records prototype work verified on **`/dev-skia-test`** only, and **`edit.tsx`** mounts **`<KonvaCanvas>`**. Task 1.3 is complete and canvas-agnostic. (This header previously read "Interactive Skia Canvas... In Progress (Task 1.1 Rewired & Verified)"; corrected 2026-09-23.)*

Architectural Correction & Implementation Log: Dual-Reconciler Shared-Value Bridge, Barcode Module Rendering & QR 1:1 Square Lock

Superseded (Tasks 1.1 & 1.2), noted 2026-09-23: Everything in this entry was built and verified on the Skia editor prototype (**`/dev-skia-test`**). Task 1.4, logged the same day, kept the live editor on Konva, so none of it is on the live editor path. The record is kept as history. The live Konva equivalents (pinned-origin resize, QR square lock, headroom clamping) are logged under the Standalone Bugfixes above.

- **Scope:** Skia canvas rendering tree, symbology module generation & Reanimated shared-value bridges (**`src/components/editor/skia-canvas.tsx`**, **`src/components/editor/skia-element-renderer.tsx`**, **`src/app/dev-skia-test.tsx`**).
- **Date Completed:** 2026-09-18
- **Retraction & Root Cause Clarification:**
 - **Task 1.1 & Task 1.2 Part B Previous Verification Retraction:**

- Task 1.1 was previously reported as complete and verified based on `SkiaElementNode`'s internal Reanimated math in isolation.
 - However, an architectural investigation confirmed that `SkiaCanvas` rendered static `<Group>` props inside `<Canvas>` and had never actually mounted `SkiaElementNode` into the active Skia Fiber tree. As a result, live GPU scaling was completely disconnected from touch gestures on screen.
- **Barcode Solid Black Block Root Cause (Cause 1):**
 - `barcodeBarsForMode` returns `BarcodeBar[] = { x: number, width: number }[]` (normalized coordinates for each dark bar).
 - `SkiaBarcode` in `skia-element-renderer.tsx` incorrectly treated `validBars` as boolean module flags (`if (validBars[i])`), causing the loop to always evaluate truthy and collapse all bars into a single solid black rectangle spanning the entire width.
- **QR Code Aspect Distortion Root Cause:**
 - While QR matrix generation was correct, `widthResizeGesture` and `heightResizeGesture` allowed independent width and height mutations without 1:1 aspect constraint, causing GPU `scaleX` to distort square QR modules into wide non-scannable rectangles.
- **Skia Font Measurement (Cause 2):**
 - `font.getTextWidth()` was called on `SkFont`. While present in the `.d.ts` file as `@deprecated` (allowing TypeScript compilation), it was unmapped in the runtime JSI bridge.
- **What Was Done & Architectural Fix:**
 1. **Dual-Reconciler Shared-Value Bridge in `skia-canvas.tsx`:**
 - Created `getOrCreateElementSharedState` maintaining a persistent map of Reanimated shared values (`transX`, `transY`, `curWidth`, `curHeight`, `isInteracting`).
 - **Inside `<Canvas>`:** Mounted `<SkiaElementNode>` for every element, binding GPU `outerTransform` (translation + center-pivot rotation) and `contentTransform` (GPU live scale) directly to the UI thread shared values.
 - **Outside `<Canvas>`:** Mounted transparent `<ElementGestureNode>` views with 44x44pt vector handle hitboxes driven by the exact same shared values.
 2. **QR Code 1:1 Square Lock (Option a):**
 - Enforced synchronous dual-handle driving: dragging the East handle or South handle calculates `maxAllowed = Math.min(canvasWidth - left, canvasHeight - top)` upfront, capping the drag range at physical canvas headroom without jitter.
 - Synchronously updates `curWidth.value = newSize` AND `curHeight.value = newSize` ($s_x = s_y$) on every frame, preserving perfect 1:1 square geometry.
 3. **Discrete Barcode Module Rendering in `skia-element-renderer.tsx`:**
 - Rewrote `SkiaBarcode` to map `BarcodeBar[]` normalized `{ x, width }` rectangles directly:
$$x_i = \text{bar.x} \times \text{widthPx}, \quad w_i = \text{bar.width} \times \text{widthPx}$$
 4. **Skia Font API Upgrade:**
 - Upgraded all `font.getTextWidth()` call sites in `SkiaTextElement` and `SkiaBarcode` to modern `font.measureText(text).width`.

5. Handle Clearance Padding:

- Added baseline clearance padding so bottom South handles never collide with or obscure HRI text.

• Verification & Multi-Element Test Matrix Results:

- `npx tsc --noEmit`: 0 errors.
- `npm run test:labelx`: 100% PASS.
- `src/lib/editor/__tests__/skia-scaling-engine.test.ts`: 100% PASS across all discrete barcode modes, QR matrix geometry, and dual-handle square-lock constraints.
- **On-Device /dev-skia-test Matrix:**
 - **Text (el-text-1)**: East drag scales text live on GPU; South drag disabled (auto-height); releases with crisp font commit. (PASS)
 - **Barcode (el-barcode-1)**: Discrete vertical bars rendered crisply; East drag expands width with zero vertical jump; HRI text stays centered with clean South handle clearance. (PASS)
 - **Shape (el-shape-1)**: Independent East width and South height resizing with constant stroke width. (PASS)
 - **QR Code (el-qr-1)**: Dragging East or South handle synchronously drives both dimensions in a locked 1:1 square ratio; respects canvas boundary without stretching. (PASS)
 - **Translation (All Elements)**: 1:1 smooth translation with GPU live tracking and zero release jumping. (PASS)

Task 1.3: Modernize Template Resizing Engine (`src/lib/element-sizing.ts`)

- **Scope:** Template resizing and aspect-ratio-aware layout scaling (`src/lib/element-sizing.ts`, `src/lib/editor/__tests__/template-resizing.test.ts`).
- **Date Completed:** 2026-09-18
- **Problem & Root Cause:**
 - When resizing label stock dimensions (e.g. $50 \times 50 \rightarrow 50 \times 25$ mm or $50 \times 30 \rightarrow 70 \times 30$ mm), `scaleDocumentToSize` previously multiplied all coordinates and dimensions linearly by (s_x, s_y) .
 - Anisotropic scaling squashed multi-line text into truncated lines, distorted square QR codes into wide non-scannable rectangles, and reduced barcodes to unreadable thin ribbons.
- **What Was Done & Architectural Fix:**
 - 1. Dynamic Text Height Reflow:**
 - For `text` and `degrees`, scaled font size by $\min(s_x, s_y)$ (≥ 4 pt) and dynamically re-evaluated multi-line height via `computeTextElementHeightMm`.
 - Preserved full multi-line content without silent state truncation; correctly flags `overflowed = true` if reflowed text exceeds new canvas headroom.
 - 2. 1:1 QR Square Invariant:**
 - Computed isotropic size `size = min(scaled.width, scaled.height)` with `width == height`, preserving QR matrix squareness.
 - 3. Optical Scan Proportion & Ceiling Guard for Barcodes:**
 - Enforced a 6 : 1 maximum width-to-height aspect ratio ceiling (`height  $\geq$  width/6`) on width expansion, with an absolute 3.5 mm scannability floor clamped within canvas headroom.

4. Perimeter Border Locking:

- Locked `border` elements to $(0, 0, W_{\text{new}}, H_{\text{new}})$ with `lockMovement: true` and uniform stroke scaling.

• Verification & Quality Gate Results:

- `npx --yes tsx --tsconfig tsconfig.json src/lib/editor/__tests__/template-resizing.test.ts`: 100% PASS (aggressive $50 \times 50 \rightarrow 50 \times 15$ mm reduction, 70×30 mm width expansion, and 60 mm barcode on 5 mm headroom tension test).
 - `src/lib/editor/__tests__/skia-scaling-engine.test.ts`: 100% PASS.
 - `npx tsc --noEmit`: 0 errors.
 - `npm run test:labelx`: 100% PASS.
-

Task 1.4: Architecture Alignment: Editor Canvas Stabilized on Konva; Print/Export Decoupled to Headless Skia

- **Scope:** Editor stability and architectural decoupling.
 - **Date Completed:** 2026-09-18
 - **Architectural Decision:**
 - The live interactive editor remains permanently on `<KonvaCanvas>` (`src/components/editor/konva-canvas.tsx`, `konva-transformer.tsx`), ensuring zero gesture regressions, buttery 120fps direct dragging, and instant responsive resizing.
 - The direct print/export rasterizer is completely decoupled to headless Skia in Phase 4 (no shared rendering code, no gesture baggage, pure headless bitmap generation).
-

Phase 2: Integer-Module Optical Barcode & 2D Symbology Engine (Canvas-Agnostic)

Task 2.1: Integer Module Snapping Engine (`src/lib/barcode/barcode-snapping.ts`)

- **Scope:** Mathematical integer hardware dot quantization and quiet zone enforcement engine (`src/lib/barcode/barcode-snapping.ts`, `src/lib/barcode/__tests__/barcode-snapping.test.ts`).
- **Date Completed:** 2026-09-19
- **Integration / Wiring Status:**
 - **Engine Status:** Built, mathematically proven, and 100% test-verified in `src/lib/barcode/barcode-snapping.ts`.
 - **Live Canvas Wiring: Resolved by Task 2.3 (2026-09-19)** — see the Task 2.3 entry below. *Original note at time of 2.1:* The live Konva renderer (`src/components/editor/element-renderer.tsx`) currently still uses the legacy floating-point `bar.x * widthPx` rendering. Wiring the snapped integer dots and authentic matrices into `element-renderer.tsx` is the explicit deliverable of **Task 2.3** (after encoders in Task 2.2 are complete).
- **What Was Done:**
 1. Implemented `snap1DBarcodeModules`:
 - Quantizes narrow-bar width (X -dimension) to exact integer hardware dots (1, 2, 3, ... dots) at target printer resolutions (203 & 300/304 DPI).

- Enforces ISO/IEC standard $10\times$ module width quiet zones on left and right margins for 1D barcodes.
 - Automatically centers the quantized barcode within the container width to eliminate fractional sub-pixel jitter.
 - Computes optical scannability grading (**optimal**, **marginal**, **sub-optical**) based on the 0.25 mm commercial laser threshold.
2. Implemented **snap2DMatrixToHardwareDots**:
- Quantizes 2D matrix cells (QR, DataMatrix, PDF417) to discrete $N \times N$ integer dot modules.
- **Verification & Quality Gate Results:**
 - `npx tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/barcode-snapping.test.ts`: 100% PASS (6/6 tests passing: conversion helpers, integer dots, quiet zones, 300 DPI scaling, container centering, 2D matrix snapping).
 - `npx tsc --noEmit`: 0 errors.
-

Task 2.2: Authentic PDF417 & DataMatrix Encoders (`src/lib/barcode/pdf417.ts`, `src/lib/barcode/datamatrix.ts`)

- **Scope:** Canvas-agnostic ISO/IEC standards-compliant 2D matrix symbology encoders (`src/lib/barcode/pdf417.ts`, `src/lib/barcode/datamatrix.ts`, `src/lib/barcode/__tests__/encoders.test.ts`).
- **Date Completed:** 2026-09-19
- **What Was Done:**
 1. **ISO/IEC 15438 PDF417 Multi-Row 2D Symbology Encoder** (`src/lib/barcode/pdf417.ts`):
 - Implemented Text Compaction (Uppercase, Lowercase, Mixed, Punctuation sub-modes), Numeric Compaction, and Byte Compaction.
 - Implemented Reed-Solomon Error Correction Code (modulo 929 arithmetic over $GF(929)$ with primitive root 3) across error levels ECC 0 through ECC 8.
 - Implemented full row assembly with ISO-compliant 17-module Start pattern (**11111111010101000**), Left row indicator, 17-module data codewords, Right row indicator, and 18-module Stop pattern (**111111101000101001**).
 - Returns boolean matrix grid `boolean[][]` along with row and column dimensions.
 2. **ISO/IEC 16022 DataMatrix ECC 200 2D Symbology Encoder** (`src/lib/barcode/datamatrix.ts`):
 - Implemented ASCII compaction mode, 253-pad byte randomization, and $GF(256)$ Reed-Solomon polynomial division with primitive polynomial $x^8 + x^5 + x^3 + x^2 + 1$ (0x12D).
 - Implemented Utah placement algorithm mapping interleaved data and error correction codewords to symbol coordinates.
 - Constructed authentic alignment patterns: solid dark bottom and left "L" finder patterns, and alternating top and right timing clock tracks (with ECC 200 top-right light corner verification).
 - Supports both square symbols (from 10×10 up to 144×144) and rectangular formats (e.g. 8×18 , 12×26 , 16×36 , etc.).
- **Verification & Quality Gate Results:**

- **Independent Engine Verification (`src/lib/barcode/__tests__/independent-decoder-verification.test.ts`): 100% PASS**
 - Tool: `@zxing/library` (`DataMatrixReader` and `PDF417Reader`).
 - DataMatrix ECC 200: Successfully generated matrices and independently decoded back exact strings across numeric ("`0123456789`"), alphanumeric ("`SEZ-PRINT-AUTHENTIC-2026`"), rectangular ("`BATCH-9941`"), and date ("`EXP:2028-12-31`").
 - PDF417 ISO/IEC 15438: Successfully generated matrices and independently decoded back exact strings across short alphanumeric ("`TEST1234`"), mixed-case with spaces and numbers ("`Hello World 12345`"), enterprise tag ("`SEZ-PRINT-ENTERPRISE-2026`"), and postal tracking ("`Tracking# 9400 1000 0000 0000 00`").
 - `npx --yes tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/encoders.test.ts`: 100% PASS (square and rectangular DataMatrix ECC 200, L-finder bars, clock tracks, PDF417 start/stop guard patterns, and variable ECC levels verified).
 - `npx --yes tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/barcode-snapping.test.ts`: 100% PASS.
 - `npx tsc --noEmit`: 0 errors.
 - **Notes & Next Step:** Proceeded to Task 2.3.
-

Task 2.3: Update Barcode & 2D Rendering in `element-renderer.tsx` & Symbology Core

- **Scope:** Wiring hardware-quantized integer module snapping and authentic 2D matrix symbologies (PDF417 and DataMatrix) into the live Konva canvas renderer (`src/components/editor/element-renderer.tsx`), completely eliminating legacy floating-point jitter and fake pseudo-random matrices.
- **Date Completed:** 2026-09-19
- **What Was Done:**
 1. **Integer Module Snapping & Quiet Zone Integration in `BarcodeContent`:**
 - Exported `barcodeModulesForMode` in `src/lib/barcode-code128.ts` to provide raw module sequences for all supported 1D modes (`CODE-128`, `CODE-39`, `ITF`, `UPC-A`, `EAN-13`, `EAN-8`).
 - Wired `snap1DBarcodeModules` into `BarcodeContent` in `src/components/editor/element-renderer.tsx`, replacing the old floating-point `bar.x * widthPx` path math with hardware-quantized, integer-dot-aligned bar positions (`snapped.bars`).
 - Enforced ISO/IEC 10× quiet zones on left and right borders and automatic container centering (`snapped.offsetXmm`), eliminating edge-bleed and dot-jitter on thermal printheads.
 2. **Authentic 2D Matrix Rendering in `QrcodeContent`:**
 - Wired `encodeDataMatrix` (ISO/IEC 16022 ECC 200) for DataMatrix elements, rendering real square and rectangular matrix modules in single-path SVG with clean quiet zones.
 - Wired `encodePdf417` (ISO/IEC 15438) for PDF417 elements, rendering authentic multi-row cluster modules with proper 3 : 1 row aspect ratios and start/stop guard patterns.
 - Preserved single-path vector rendering for standard QR codes (`generateQrMatrix`).

3. Complete Removal of `pseudoMatrix`:

- Completely removed the `pseudoMatrix` mock generator function from `element-renderer.tsx`. Zero fake barcode generation remains in the repository.

• Verification & Quality Gate Results:

- `npx --yes tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/independent-decoder-verification.test.ts`: 100% PASS across ZXing `DataMatrixReader` and `PDF417Reader`.
- `npx --yes tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/encoders.test.ts`: 100% PASS.
- `npx --yes tsx --tsconfig tsconfig.json src/lib/barcode/__tests__/barcode-snapping.test.ts`: 100% PASS.
- `npx --yes tsx --tsconfig tsconfig.json src/lib/editor/__tests__/template-resizing.test.ts`: 100% PASS.
- `npm run test:labelx`: 100% PASS.
- `npx tsc --noEmit`: 0 errors.

• Notes & Next Step: Proceeded to Task 2.4.

Task 2.4: Real-Time Scannability Preflight Inspector

- **Scope:** Building a real-time ISO/IEC 15416 preflight scannability engine (`src/lib/barcode/scannability-inspector.ts`) and integrating real-time diagnostic badges, metrics, and auto-optimization into the editor property panel (`src/components/editor/barcode-property-panel.tsx`).
- **Date Completed:** 2026-09-19
- **What Was Done:**
 - Scannability Engine (`src/lib/barcode/scannability-inspector.ts`):**
 - Implemented `inspect1DBarcodeScannability(mode, content, widthMm, heightMm, dpi)` returning a full preflight report with status (`optimal`, `marginal`, `sub-optimal`, `invalid`), optical score (0–100), ISO grade (A, B, C, F), and concrete diagnostics.
 - Implemented dual-axis `computeOptimalDimensionsMm(mode, content, currentHeightMm, dpi)` computing width for 2-dot modules and height (≥ 3.5 mm floor, $\geq 15\%$ width, ≥ 5 mm).
 - UI Property Panel Integration (`src/components/editor/barcode-property-panel.tsx`):**
 - Built `ScannabilityInspectorCard` with live status pill (● Optimal / ● Marginal / ● Sub-Optimal), diagnostic metrics grid (X -dimension, quiet zone, physical height), diagnostic warning bullet list, and "Auto-Optimize Dimensions" button.
- **Verification & Quality Gate Results:**
 - `src/lib/barcode/__tests__/scannability-inspector.test.ts`: 100% PASS (6/6 tests passing).
 - `npx tsc --noEmit`: 0 errors.
- **Notes & Next Step:** Proceeded to Barcode Bounding Box Sizing Alignment vs WePrint.

Bugfix: Barcode Selection/Bounding Box Sizing Mismatch vs WePrint

- **Scope:** Eliminating empty internal padding and vertical dead space inside the barcode selection bounding box to match WePrint's tight fit (`src/components/editor/element-renderer.tsx`, `src/lib/element-sizing.ts`, `src/components/editor/types.ts`).
- **Date Completed:** 2026-09-19
- **Problem & Root Cause:**
 - **Horizontal Dead Space:** In `snap1DBarcodeModules`, passing `includeQuietZone = true` caused 20 modules of blank quiet zone to be rendered *inside* the element bounding box, and `offsetXMm` centered the barcode within oversized containers (e.g. 37.6mm–47.2mm from `fitBarcodeDefaults`), leaving 7.5mm (40% of the box) in dead gutters on left and right.
 - **Vertical Dead Space ("above the bars"):** In `BarcodeContent`, the outer container used `justifyContent: 'center'`, which vertically centered the bars and text inside inflated containers, pushing the top of the bars down away from the top dashed border.
 - **Reference WePrint Measurements (`media_1789801439229.jpg`):**
 - Red dashed box: 255 × 101 px (aspect ratio 0.396).
 - Top gap: **0 px** (bars start on row 240, top border is row 240).
 - Left gap: **2 px** (0.8%).
 - Right gap: **1 px** (0.4%).
 - Bars height: 71 px (70.3%), text height: 17 px (16.8%), gap: 7 px (6.9%), bottom handle clearance: 6 px (5.9%).
- **What Was Done:**
 - 1. Tight Barcode Rendering (`element-renderer.tsx`):**
 - Switched `snap1DBarcodeModules(rawModules, widthMm, 203, false)` to render with `includeQuietZone = false` inside the element bounding box. The first bar starts at $x = 0$ (left border) and the last bar reaches $x = \text{widthPx}$ (right border).
 - Changed outer container `justifyContent` to `'flex-start'`.
 - Pinned the top of the bars at $y = 0$ (touching top dashed border).
 - Fixed `barsHeight = Math.max(2, heightPx - labelHeight)` so bars and text consume 100% of `heightPx` with zero dead space above the bars.
 - 2. Tight Default Sizing (`element-sizing.ts` & `types.ts`):**
 - Updated `fitBarcodeDefaults` to set natural tight barcode proportions matching WePrint (≈ 26 mm width, ≈ 10 mm height, aspect ratio ≈ 0.39).
 - Updated `DEFAULT_BARCODE_STATE` to 26 mm × 10 mm, `fontSize: 8`.
- **Verification & Quality Gate Results:**
 - `scannability-inspector.test.ts`: 100% PASS.
 - `barcode-snapping.test.ts`: 100% PASS.
 - `encoders.test.ts`: 100% PASS.
 - `independent-decoder-verification.test.ts`: 100% PASS (ZXing).
 - `template-resizing.test.ts`: 100% PASS.
 - `npm run test:labelx`: 100% PASS.
 - `npx tsc --noEmit`: 0 errors.

Editor Feature: Multi-Select ("Multiple" Mode) — Konva Editor

Status: Implementation in progress (rewrite underway). Spec: `ARCHITECTURE_REDESIGN_PLAN.md` → "Editor Feature Track: Multi-Select".

Multi-Select: Current State (recorded 2026-09-23)

- **Scope:** `src/app/edit.tsx`, `src/components/editor/konva-canvas.tsx`, `src/components/editor/konva-transformer.tsx`, `src/lib/editor/selection.ts`, `src/lib/editor/resize-policy.ts`, `src/components/editor/multi-select-property-panel.tsx`.
- **Commits so far:** `720aab6` "multi select fixed", `f4a21ca` "multiple selection blink fix". There is no per-change log for these commits; this entry records the state at the time of the documentation sync.
- **Locked Technical Model:**
 1. **Shared scale factor, reusing single-element resize logic:** a group resize yields one shared scale (`groupScaleXSv` / `groupScaleYSv` in `konva-transformer.tsx`). Each member's box comes from `resizeMemberByScale` — the same function single-element resize uses (`boundBoxMm` delegates to it).
 2. **No origin movement during resize:** every member keeps its gesture-start `left` / `top`, live and on commit. Only `width` / `height` change.
 3. **Shared capped ratio:** `sharedScaleLimits` (called from `edit.tsx` at resize start) sets `groupScaleMinSv` / `groupScaleMaxSv` to the highest per-member minimum scale and the lowest per-member canvas-edge scale. The whole group stops together when any member reaches its `minMm` or the canvas edge. Members with no resize behaviour for the dragged handle are excluded from the cap.
- **Verification & Quality Gate Results (run 2026-09-23):**
 - `npx tsx --tsconfig tsconfig.json src/lib/editor/__tests__/selection.test.ts`: 11/11 PASS.
 - `src/lib/editor/__tests__/resize-member-by-scale.test.ts`: 7/7 PASS.
 - `src/lib/editor/__tests__/multi-transform-verify.test.ts`: 4/4 PASS (shared move delta live = commit; origins stationary on east→south; group member = solo `resizeMemberByScale`; group stops at `minMm`).
 - `npx tsc --noEmit`: not run as part of this entry.
- **Outstanding Verification (on-device, not yet done):** live equals commit with no release jump; group stops together at both caps with mixed types; square-locked QR inside a one-axis group resize; ruler and chrome follow union bounds; no selection blink or unintended drag on touch-down add; mode-off clears selection and primary promotion; property panel and align actions as one undo step.
- **Notes & Next Step:** Finish the rewrite, then run the on-device matrix above and log results here before ticking the checklist items.

Documentation & Investigation Log

2026-09-23 — Documentation Sync Pass + Independent Printer SDK Investigation

- **Scope:** `ARCHITECTURE_REDESIGN_PLAN.md`, `CHECKLIST.md`, `PROGRESS.md`; new `SDKS.md`. Documentation only — no app code changed.
- **What Was Done:**
 1. **Printer SDK investigation → `SDKS.md` (new).** Read the five vendor SDKs directly (`javap` on JARs/AARs, `nm -D` / `strings` / SHA-1 on `.so`, vendor demos and PDFs, our Kotlin wrappers,

and merged native libs in the build output). Recorded per-SDK contract, transports and risks; a ranked comparison table; cross-cutting findings; and open questions for product. Nothing was hardware-tested, and hardware-only facts are marked as such.

2. **Phase 6 rewritten** (plan and checklist) from a 3-task placeholder into a 7-task spec based on `SDKS.md`: build/packaging hygiene, universal driver contract, single TSPL/ESC-POS generator plus thin adapters, capability discovery, per-driver defect fixes, calibration store with migration, and wizard. Also corrected stale "current standing" claims: per-MAC offsets are already persisted in `printer-store.ts`, and head alignment already lives in `print-spec.ts` profiles.
 3. **Multi-select section added** to all three docs: in-progress status, the locked technical model, and the outstanding verification list. The unit tests were run to record their current pass state.
 4. **Barcode bounding-box bugfix**: added the missing checklist line under Phase 2 ("Bugfix (post-2.4)") to match its progress entry.
 5. **QR entries merged, not split**. Checklist "QR Code Vector Rendering & Resize Bleed" and progress "QR Code Single-Path Vector Rendering & Fluid 120fps Gesture Scaling" describe one change set: same date, same files (`element-renderer.tsx`, `konva-transformer.tsx`, `resize-policy.ts`), and one verification run. "Resize bleed" is the headroom-clamp part of that work, not a separate fix with its own evidence. The checklist line now uses the progress title and lists all three files; the progress entry has a mapping note.
 6. **Phase 1 contradiction resolved**. Tasks 1.1–1.2 are marked superseded by Task 1.4 in the plan and checklist (kept, struck through, with their Konva equivalents). Task 1.3 stays complete as canvas-agnostic. The plan's Phase 1 heading, roadmap line, architecture diagram editor node, and benchmark rows 1 and 3 no longer describe a Skia editor. The progress Phase 1 header status was corrected, and the Skia prototype entry is annotated as superseded.
 7. **Other consistency fixes**: plan Phase 2 checkboxes ticked to match the checklist and progress log; plan benchmark row 9 updated to the real calibration state; plan Phase 1 test-file name corrected (`template-resizing.test.ts`); the Task 2.1 "PENDING Task 2.3" note marked resolved.
 8. **Missing progress entry found**: checklist "Vertical Resize Height Jitter" (added in commit `0675688`) never had a progress entry. A retroactive entry was added from the commit diff, explicitly marked as having no recorded verification.
- **Verification**: Cross-checked every checklist line against a progress entry and every plan checkbox against the checklist. Multi-select unit tests: 22/22 PASS.
 - **Notes & Next Step**: The product decisions listed in `SDKS.md` → "Open Questions for Product" (native-library collision, Label X licence key and data upload, Tez allowlist bypass, Dev width cap) block parts of Phase 6 Tasks 6.1 and 6.5.